

EXPERIMENT 4

AIM: Write a program for Hamming Code generation for error detection and correction.

SOLUTION:

Hamming Code is an **error-control technique** used to **detect and correct a single-bit error** in transmitted data.

In this program:

1. The program accepts **4 data bits** from the user.
2. It adds 3 parity bits at positions **1, 2, and 4**, which are powers of 2.
3. The parity bits are calculated using **even parity**.
4. The program generates a **7-bit Hamming Code**.
5. The user enters an error position:
 - ✓ 0 → No error
 - ✓ 1–7 → One bit is changed to simulate an error during transmission.
6. At the receiver, the program checks the parity bits.
7. The parity results are used to find the **position of the error**.
8. If an error is found, the program **corrects the erroneous bit**.
9. Finally, the program displays the **corrected Hamming Code and original data**.

Source Code:

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int h[7];
    int error;
    int p1, p2, p4;
    int error_position;
    int i;

    clrscr();

    /* Enter 4 data bits */
    printf("Enter 4 data bits: ");
    scanf("%d%d%d%d", &h[2], &h[4], &h[5], &h[6]);

    /* Calculate parity bits - Even parity */
    h[0] = h[2] ^ h[4] ^ h[6]; /* P1 */
    h[1] = h[2] ^ h[5] ^ h[6]; /* P2 */
    h[3] = h[4] ^ h[5] ^ h[6]; /* P4 */

    /* Display Hamming Code */
    printf("Hamming Code: ");

    for (i = 0; i < 7; i++)
    {
        printf("%d", h[i]);
    }

    /* Enter error position */
    printf("\n\nEnter error position (0 for no error, 1-7 for error): ");
    scanf("%d", &error);
```

```

/* Add error */
if (error >= 1 && error <= 7)
{
    h[error - 1] = !h[error - 1];

    printf("Error added at position %d\n", error);
}
else if (error == 0)
{
    printf("No error added\n");
}
else
{
    printf("Invalid position\n");
    getch();
    return 0;
}

/* Display received code */
printf("Received Code: ");

for (i = 0; i < 7; i++)
{
    printf("%d", h[i]);
}

/* Check parity bits */
p1 = h[0] ^ h[2] ^ h[4] ^ h[6];
p2 = h[1] ^ h[2] ^ h[5] ^ h[6];
p4 = h[3] ^ h[4] ^ h[5] ^ h[6];

/* Find error position */
error_position = p4 * 4 + p2 * 2 + p1;

if (error_position == 0)
{
    printf("\nNo error detected");
}
else
{
    printf("\nError detected at position: %d", error_position);

    /* Correct the error */
    h[error_position - 1] = !h[error_position - 1];

    printf("\nCorrected Code: ");

    for (i = 0; i < 7; i++)
    {
        printf("%d", h[i]);
    }
}

/* Display original data */

```

```
    printf("\nOriginal Data: %d%d%d%d\n",
           h[2], h[4], h[5], h[6]);

    getch();

    return 0;
}
```

Output: